

Final Exam Topics

20. Databases - design and query

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Databases – design and query

Relational data model, entity-relationship model and its transformation into the relational data model. Relational algebra, SQL. The procedural extension of SQL (PL/SQL or PSM). Designing relational database schemas, normal forms, decompositions.

1 Relational data model, entity-relationship model and its transformation into the relational data model

1.1 Relational data model

Relational data model: manages a collection of data.

Relation: the specification of the collection (table).

Data model: a higher-level representation of the concepts, relationships, and activities of reality, which specifies, both for the computer and the user, what the data looks like. A notation serving to describe data.

Its parts:

1. the structure of the data
2. the operations that can be performed on the data
3. the constraints placed on the data

Other concepts:

- Relation schema: *relation name(row type)*, $R(A_1, \dots, A_n)$
- Occurrence: an instance, *finitely many* rows corresponding to the row type, $\{t_1, \dots, t_m\}$, where $t_i = \langle v_i1 \dots v_in \rangle$
- Attributes: data type : row type, $\langle \text{attr.name}_1 : \text{value type}_1, \dots \rangle$
- Keys: A simple key consists of 1 attribute, a composite one of several; there cannot be two different rows in the relation with the same key.
- Foreign key: $R(A_1, \dots, A_m)$ relation, $X = \{A_{i_1}, \dots, A_{i_k}\}$ key. $S(B_1, \dots, B_n)$ relation, $Y = \{B_{j_1}, \dots, B_{j_k}\}$ foreign key referencing X in the given attribute order: B_{j_1} to A_{i_1} , etc.
- Referential integrity: a constraint on the joint occurrence of the two tables. If $s \in S$ is a row, then $\exists t \in R$ row: $s[B_{j_1}, \dots, B_{j_k}] = t[A_{i_1}, \dots, A_{i_k}]$.

The relational data model is advantageous from several points of view, due to which it became widespread and refined. The data model contains a simple and easily understandable structural part. The natural tabular form does not have to be explained, and is more applicable. In the relational model the conceptual-logical-physical levels are completely separated, with a high degree of logical and physical data independence. The user works at a high level, with concepts close to them (the implementation is hidden). It has a theoretical foundation, several abstract handling languages exist, for example relational algebra (the automatic and efficient query optimization of SQL is based on this). Its operational part is a simple user interface, the standard SQL.

Structure of a relational database: The database is essentially a set of relations. The set of corresponding relation schemas gives the database schema (denoted by the double-barred $\mathbb{R}$), $\mathbb{R} = \{R_1, \dots, R_k\}$. The associated occurrences are the database occurrence. Content of the occurrence: the occurrences of the individual relations. This is the conceptual level, that is, the conceptual model. Physical model: it represents the table in some file structure (for example in a serial file). Relational database managers index; an indexing method is e.g. the B+ tree.

1.2 Entity-relationship model

Its elements:

- Entity sets: a collection of similar entities
- Attributes: observable properties, observed values, properties of entities
- Relationships: relationships with other entity sets
- Schema: $E(A_1, \dots, A_n)$ entity set schema, E name, A_i property, $DOM(A_i)$ the set of possible values, e.g. *teacher(name, department)*
- Occurrence: Concrete entities, $E = \{e_1, \dots, e_m\}$, $e_i(k) \in DOM(A_k)$ the set of entities. They cannot agree on every attribute $\rightarrow$ (that is, all properties together form a superkey), minimal superkey = key.
- $K(E_1, E_2)$ binary relationship, $K(E_1, \dots, E_p)$ the schema of the relationship. K is the name of the relationship, E_i the schemas of the entity sets, a multi-way relationship if $p > 2$. E.g. *teaches(teacher, subject)*. A multi-way relationship can be rewritten into the corresponding number of binary relationships, a 3-way one into 3, etc.
- An occurrence of a relationship with schema $K(E_1, \dots, E_p)$, $K = \{(e_1, \dots, e_p)\}$ a set of entity p-tuples, where $e_i \in E_i$. The constraints placed on the occurrences of the relationship determine the type of the relationship.

Diagram: representation of entity sets, relationship types, entities.

Roles: an entity set relates to itself

Relationship attribute: a joint function of two entity sets, but of neither one separately

Entity set: the properties belonging to the primary key are underlined

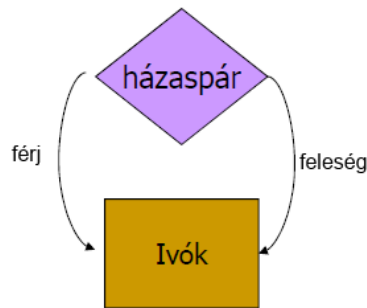


Figure 1: Roles

Types of relationships (let us take a $K(E_1, E_2)$ binary relationship as a basis):

- many-to-one: in the occurrences of K , each entity in E_1 can be associated with at most 1 entity in E_2 , e.g. *born(name, country)*
- many-to-many: it has no constraint, each entity in E_1 can be associated with several entities in E_2 , and conversely, each entity in E_2 can be associated with several entities in E_1 , e.g. *learns(student, language)*
- one-to-one: many-to-one and one-to-many, that is, each entity in E_1 can be associated with at most one entity in E_2 , and conversely, each entity in E_2 can be associated with at most one entity in E_1 , e.g. *married couple(man, woman)*

There can also be several relationships between two entity sets.

Subclass: “isa” = “is-a”, inheritance, a special one-to-one relationship. For every entity in E_1 there is one in E_2 , e.g. *isa(boss, employee)*

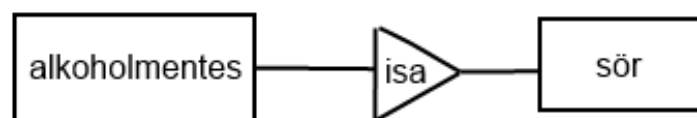


Figure 2: Subclass

Key: marked with underlining; in a composite key several attributes are underlined

Superkey: A superkey of the entity set is an identifier, that is, a set of properties about which it can be assumed that no two different entities appear in the occurrences of the entity set that agree on these properties. All properties together are always a superkey.

Referential integrity: marked with a round ending, a constraint

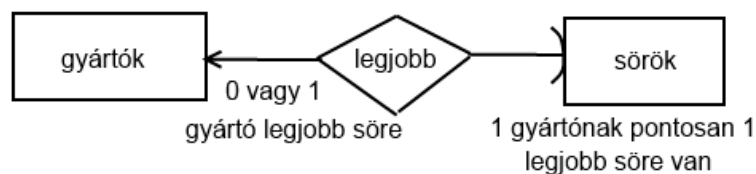


Figure 3: Referential integrity

Weak entity set: A rectangle with a double contour. By itself it cannot be unambiguously identified.

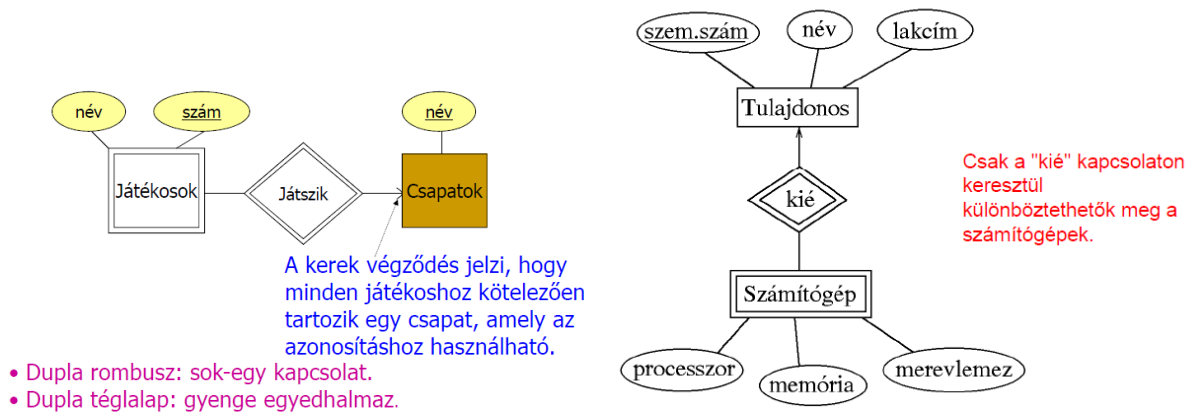


Figure 4: Weak entity set

Design principles:

- faithful modeling: the appropriate properties should belong to the entity classes; for example the teacher's name should not be among the student's properties
- avoiding redundancy: the $index(etr-code, address, subject, date, grade)$ is a bad schema, because the address is repeated as many times as the student has exam grades; instead it is worth introducing 2 schemas: $student(etr-code, address)$, $exam(etr-code, subject, date, grade)$.
- simplicity: do not introduce entity classes unnecessarily; for example instead of $calendar(year, month, day)$ rather use a date property at the appropriate place
- property or entity class: for example instead of an exam grade class use a grade property.

1.3 Transformation of the entity-relationship model into the relational data model



KÖNYV (könyvszám, szerző, cím)
 OLVASÓ (olvasószám, név, lakcím)
 KÖLCÖN (könyvszám, olvasószám, kivétel, visszahozás)

Figure 5: Example: data model of a library

Mapping of composite attributes: for example if we want to handle the address in a (locality, street, house number) structure, then all of them have to be introduced into the schema as attributes. *Mapping of multivalued attributes:*

- Specification as single-valued. Example: for a multi-author book we list all of them in one field. Not very good, because the authors cannot be handled separately, and possibly not all of them fit in the field.

- Specification as multivalued.
 - Multiplication of rows. Staying with the book example, we introduce as many rows as there are authors. This leads to redundancy.
 - Introduction of a new table. We replace the *book(book number, author, title)* schema with the following two schemas: *book(book number, title)*, *author(book number, author)*
 - Numbering. If the order of the authors matters, then in the previous solution (new table) the author table has to be supplemented with a sequence number field.

Mapping of relationships:

- one-to-one: merging into the schema with the same key. E.g. a book can be borrowed: from the relationship schema LOAN(book number, reader number, **checkout**) we get BOOK (book number, author, title, reader number, **checkout**), READER (reader number, name, address). Here, in the relationship schema, the reader number is also a key, and we could also introduce it so that it is the key. In that case it has to be merged into READER.
- many-to-one: merging into the “many”. So following the example, if several books can be borrowed, then the book number has to be the key, and we can only merge into BOOK.
- many-to-many: a new table. If the earlier loans are also stored, then the key includes either the checkout or the return. In this case it cannot be merged into either table, a new table has to be created. The schema can be this: BOOK (book number, author, title), READER (reader number, name, address), LOAN (book number, reader number, checkout, return).

Transformation E/R model → relational data model:

- entity set schema → relation schema
- properties → attributes
- (super)key → (super)key
- occurrence of the entity set → relation
- entity → $e(A_1)...e(A_n)$ row
- $R(E_1, ...E_p, A_1, ...A_q)$ relationship schema (E_i entity set, A_j property) → $R(K_1, ...K_p, A_1, ...A_q)$ relation schema (K_i the (super)key of E_i)

We can rename to avoid having the same thing twice.

In the case of isa, we add to the special class the (super)key of the general class. To a weak entity we add the keys of the determining relationships.

Merging is possible if one of them has a foreign key to the other. Likewise when one of the relations corresponds to a many-to-one relationship, and the other is the relation of the entity set on the many side. E.g. Drinkers(name, address) and Favorite(drinker,beer) can be merged, and we get the schema Drinker1(name,address,favoriteBeer).

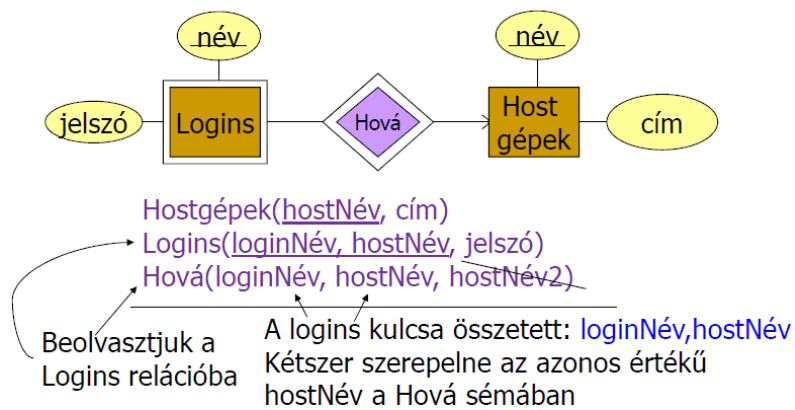


Figure 6: Rewriting of a weak entity set

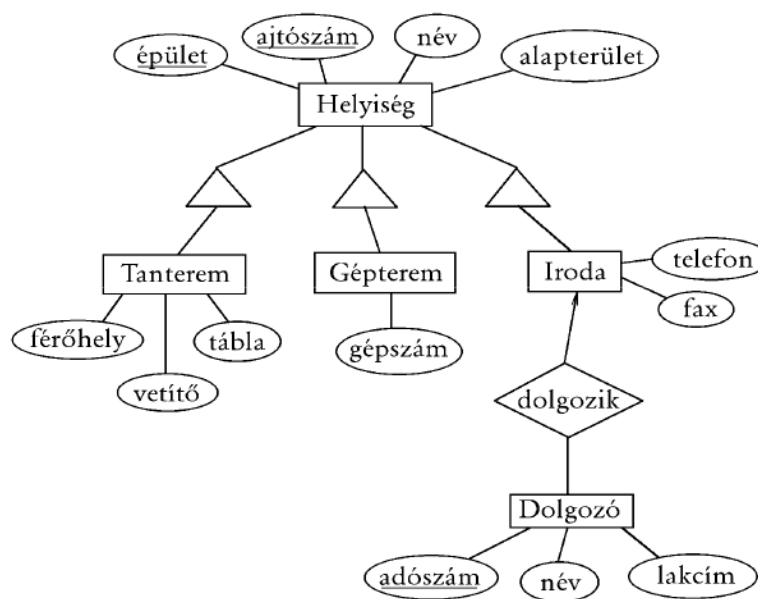


Figure 7: Example of rewriting subclasses into relations

Rewriting of subclasses:

- E/R-style: in the model let there be 1 relation for every subclass, but from the general one we add only the keys to its own attributes.

A separate table for every subtype; an entity can appear in several tables. Every entity appears in the table of the main type, plus in as many of the subtypes as it belongs to. Disadvantage: searching across several tables.

(E/K stílusú reprezentálás.)

HELYISÉG (épület, ajtószám, név, alapterület)
 TANTEREM (épület, ajtószám, férőhely, tábla, vetítő)
 GÉPTEREM (épület, ajtószám, gépszám)
 IRODA (épület, ajtószám, telefon, fax)
 DOLGOZÓ (adószám, név, lakcím, épület, ajtószám)

Figure 8: E/R style

- Null-valued: there is 1 relation in total; if the special property is not present, then we write NULL in its

place.

The union of the attributes appears in the table. The disadvantage is that there are many NULLs in the table, and we can also lose type information (for example if for the computer lab the computer count is unknown, and is therefore NULL, then the computer lab essentially falls into the category of other rooms).

(Reprezentálás nullértékekkel)
 HELYISÉG (épület, ajtószám, név, alapterület, férőhely, tábla,
 vetítő, gépszám, telefon, fax)
 DOLGOZÓ (adószám, név, lakcím, *épület*, *ajtószám*)

Figure 9: NULL-valued

- Object-oriented: 1 relation for every subclass, with all properties listed including the inherited ones. A separate table for every subtype; an entity appears in only 1 table. Disadvantages: for a combined subtype the introduction of a new subtype is necessary, and searching often goes across several tables.

(Objektumorientált stílusú reprezentálás)
 HELYISÉG (épület, ajtószám, név, alapterület)
 TANTEREM (épület, ajtószám, név, alapterület, férőhely,
 tábla, vetítő)
 GÉPTEREM (épület, ajtószám, név, alapterület, gépszám)
 IRODA (épület, ajtószám, név, alapterület, telefon, fax)
 DOLGOZÓ (adószám, név, lakcím, *épület*, *ajtószám*)

Figure 10: Object-oriented

2 Relational algebra, SQL

2.1 Relational algebra

An algebra contains operations and atomic operands.

Relational algebra: by applying operations performed on the atomic operands and on the algebraic expressions, we specify operations on the obtained relations and build expressions (the expression corresponds to the syntactic form of the question). It is therefore important that the end result of every operation is a relation, on which further operations can be specified. The atomic operands of relational algebra are the following: the variables belonging to the relations; constants, which express a finite relation. *Operations*:

- Set operations: A relation occurrence is a set consisting of finitely many rows. Thus the usual set operations can be interpreted: union (the result is a set, a row appears only once), intersection and difference can be interpreted.

R, S of the same type, the type of $R \cup S$ and $R - S$ is the same

The basic operations are union and difference; the intersection operation is derived: $R \cap S = R - (R - S)$

- Projection: it projects a given relation onto the attributes appearing in the subscript (it reduces the number of attributes). $\Pi_{list}(R)$, where $\{A_{i_1}, \dots, A_{i_k}\}$ is a listing of a subset of the attributes in R's schema. From the rows of the relation it selects the values appearing on the attributes $A_{i_1}, \dots, A_{i_k}$; if a value occurs several times, then we filter out the duplicates (to get a set).

- Selection (filtering): it selects those rows of the relation appearing in the argument that satisfy the condition appearing in the subscript. $\sigma_F(R) = \{t | t \in R \text{ and } t \text{ satisfies the condition } F\}$

The condition can be elementary or compound. Elementary: $A_i \Theta A_j$, $A_i \Theta c$, where c is a constant, and Θ is $=, \neq, <, >, \geq, \leq$. Compound: if B_1, B_2 are conditions, then $\neg B_1, B_1 \cap B_2, B_1 \cup B_2$ and the parenthesizations are also conditions.

- Natural join: of the product-like operations only this is a basic operation. The number of attributes grows. It is built on the common attribute names. $R \bowtie S$ contains those pairs of rows from R and from S that agree on the common attributes of R and S. The type of $R \bowtie S$ is the union of the two attribute sets.
- Renaming: with it we can express the product of a relation with itself. $\rho_{T(B_1, \dots, B_k)}(R(A_1, \dots, A_k))$; if we do not want to rename the attributes, then $\rho_T(R)$

In relational algebra there are the 6 basic operations listed above. This is a *minimal set*, that is, omitting any of them, it cannot be expressed with the others.

For a product-like operation we can regard the *direct product* as a basic operation, but the natural join is used. $R \times S$: every row of R and S is concatenated in pairs, to every row of the first table we append every row of the second table. In the case of the direct product (or product, Cartesian product) the equality of the attributes is of course not important. The attributes with the same name in the two or more relations, however, have to be distinguished from each other (by renaming).

Monotonicity: in the case of a monotone, non-decreasing expression, applied to a larger relation the result is also larger.

Subtraction counts as an exception among the basic operations, because it is not monotone. A consequence of this is that subtraction cannot be expressed with the other basic operations. Without subtraction it is customary to call it monotone relational algebra.

Division: on the pattern of division with remainder. The schemas of R and S are $R(A_1, \dots, A_n, B_1, \dots, B_m)$ and $S(B_1, \dots, B_m)$, the schema of $Q = R \div S$ is $Q(A_1, \dots, A_n)$. $R \div S$ is the largest (containing the most rows) relation for which $(R \div S) \times S \subseteq R$. In relational algebra: $R(A, B) \div S(B) = \Pi_{A_1, \dots, A_n}(R) - \Pi_{A_1, \dots, A_n}(\Pi_{A_1, \dots, A_n}(R) \times S - R)$. A relational algebra expression as a query language (L-language). The basic expressions are the elementary expressions, and the compound ones are the basic operations performed on them.

Evaluation of an expression: we rewrite a compound expression, proceeding from the outside inward, into an evaluation tree, leaves: elementary expressions.

Legyen R, S az $R(A, B, C)$, $S(C, D, E)$ séma feletti reláció

$\Pi_{B,D} \sigma_{A='c' \text{ and } E=2}(R \bowtie S)$

Ehhez a kiértékelő fa: (kiértékelése alulról felfelé történik)

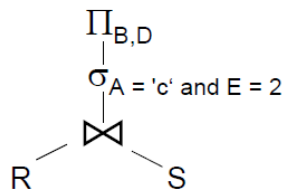


Figure 11: Expression tree

2.1.1 Extension of relational algebra

- Elimination of duplicates (δ), select distinct. It makes a set from a multiset.
- Aggregate operations and grouping (γ_{list}), group by. Aggregate functions: sum, count, min, max, avg. Here every element of the *list* is one of the following:
 - an attribute of the relation: this attribute is one of the grouping attributes (it appears after the GROUP BY).
 - an aggregate operator applied to one of the relation's attributes (this is the aggregation attribute); if we want to refer to the result of the aggregation by name, then we use an arrow and a new name.

We divide the rows of R into groups. A group contains those rows whose values belonging to the grouping attributes appearing in the list agree, that is, each distinct value of these attributes forms a group. For each

group we compute the aggregations relating to the aggregation attributes of the list. The result is one row for each group:

1. the grouping attributes and
 2. the aggregation relating to the aggregation attribute (over all rows of the given group)
- Extension of the projection operation (Π_{list}), select expr [as name]. The *list* can contain:
 - an attribute of R,
 - $x \rightarrow y$, where x, y are attribute names, and here we rename x to y,
 - $E \rightarrow y$, where E can contain attributes of R, constants, arithmetic operators and string operators, for example: $A + 5$, $C \parallel \text{'named people'}$.
 - Sorting operation (τ_{list}), order by. *list* = $(A_1, \dots, A_n)$ First we sort the rows of R by attribute A_1 . Then those rows whose value agrees on attribute A_1 , by A_2 , and so on.
This is the only operation whose result is neither a set nor a multiset.
 - Outer joins ($\bowtie$), [left | right | full] outer join. This is not a relational algebra operation, since it steps out of the model. We supplement the $R \bowtie S$ relation with the rows of R and S, writing NULL values in the missing places, thereby preserving the “dangling rows”.

In SQL, and in the extension as well, there are multisets, that is, a row can occur several times.

For the union of R and S there will be $n+m$ occurrences, for their intersection $\min[n, m]$, for the difference $\max[0, n-m]$. In the other operations we do not eliminate the duplicates.

2.2 SQL

Its main components:

- Data definition language, DDL (Data Definition Language): CREATE, ALTER, DROP
- Data manipulation language, DML (Data Manipulation Language): INSERT, UPDATE, DELETE, SELECT
SQL is primarily a query language: the SELECT statement (to obtain information from the database)
- Data control language, DCL (Data Control Language): GRANT, REVOKE
- Transaction management: COMMIT, ROLLBACK, SAVEPOINT
- Procedural extensions: Oracle PL/SQL (based on Ada), SQL/PSM (based on PL/SQL)

A relation here is a table, basically of 3 kinds: TABLE (base table, permanent), VIEW (view table), with the WITH statement (temporary work table).

The schema is specified with the CREATE statement, with the type based on the concrete realization of SQL. There are also supplementary possibilities, e.g. PRIMARY KEY. There can be only a single PRIMARY KEY in the relation, but there can be several UNIQUE; no attribute of the PRIMARY KEY can be a NULL value in any row. However, an attribute declared as UNIQUE can be NULL-valued, that is, the table can have a row where the value of the UNIQUE attribute is NULL, i.e. a missing value.

Writing relational algebra expressions with SELECT:

- SELECT list FROM product of tables WHERE cond. = $\Pi_{list}(\sigma_{cond.}(\text{product of tables}))$
- set operations: UNION, EXCEPT/MINUS, INTERSECT. A set is made from a multiset. With the ALL keyword the multiset is preserved.
- renaming: after the column name we write the new name separated by a space.

Extended operations:

- sorting: ORDER BY, it follows after every other clause, descending, ascending order.
- elimination of duplicates: select DISTINCT ...
- aggregations: SUM, COUNT, MIN, MAX, AVG in the SELECT clause. COUNT(*) is the number of rows of the result. If an aggregation also appears in the query, the attributes listed in the SELECT either appear as a parameter of an aggregate function, or also appear in the attribute list of the GROUP BY.
- grouping: GROUP BY.
- filtering of groups: HAVING. It filters groups, not individual rows. There is no restriction on the subquery. However, outside the subquery only such attributes can appear that are: either grouping attributes, or aggregated attributes. (That is, the same rules apply as for the SELECT clause).

A példa reláció: **hallgató (azon, név, tantárgy, jegy)**

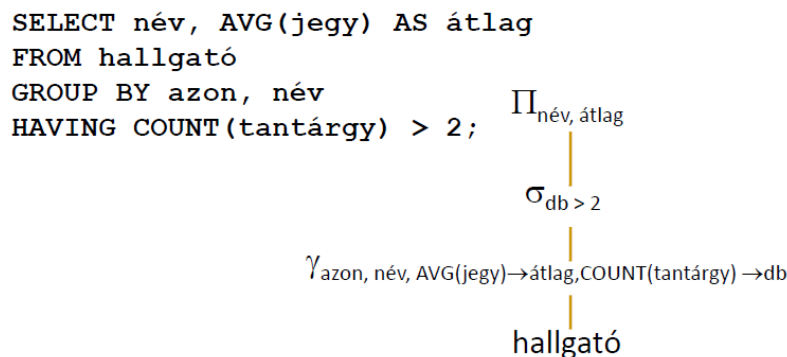


Figure 12: Example of grouping

SQL specialties in the WHERE clause:

- Can be rewritten into relational algebra: BETWEEN ... AND ..., IN(value set)
- Cannot be rewritten: LIKE for comparing strings, IS NULL (unknown or undefined value) [because of this there is 3-valued logic in SQL: true, false, unknown]

The SELECT statement consists of several parts, the parts are called clauses. There are three basic clauses:

- SELECT list (3.) – what type of row do we want to see in the result? * means: all attributes.
- FROM Relation name (1.) – the name of the table or the joining of relations (tables), or their product
- WHERE condition (2.) – which rows satisfying which conditions to select?

In the case of joining several schemas, if an attribute name occurs in several schemas, then the schema is also needed in the reference: R.A (attribute A of relation R).

We can also use *subqueries* in SQL; we achieve this by placing them in parentheses.

- In the FROM clause we can also create a temporary table this way, in which case we mostly also have to specify the name of the row variable.
- In the WHERE clause the result of the subquery can be:
 - a scalar value, that is, as if it were a constant

- a multiset of scalar values, usable in logical expressions: EXISTS, [NOT] IN, ANY/ALL (e.g. $x > \text{ANY}(\text{subquery})$).
- a full multidimensional table, usable: EXISTS, [NOT] IN

The subquery is uncorrelated if it can be evaluated independently and does not change during the outer query. It is correlated if it gets evaluated several times, with value assignment originating from a row variable outside the subquery.

Teljes SELECT utasítás
(záradékok sorrendje nem cserélhető fel)

```
SELECT [DISTINCT] ...
FROM ...
    [WHERE ... ]
    [GROUP BY ...
    [HAVING ... ] ]
    [ORDER BY ...]
```

Figure 13: SQL clauses

Joins:

- Cartesian product: R CROSS JOIN S, or “R,S”
- Natural join: R NATURAL JOIN S
- Theta join: R JOIN S ON <condition>
- Outer join: R {LEFT | RIGHT | FULL} OUTER JOIN S. LEFT preserves the dangling rows of R, RIGHT those of S, FULL all of them.

2.2.1 DML

The modifying statements do not return a result like queries, but change the content of the database. There are 3 kinds of modifying statement:

- INSERT - insertion of rows. A single row: INSERT INTO R VALUES (<attr. list>). When creating the table, if we specified default values for attributes, then if we do not give a value for it, the default will be used, otherwise NULL. Several rows: INSERT INTO R (<subquery>).
- DELETE – deletion of rows. DELETE FROM R [WHERE <condition>].
- UPDATE – modification of the values of the components of rows. UPDATE R SET <list of attribute assignments> WHERE <condition on the rows>

Transactions: Database systems are generally used by several users and processes at the same time. Transaction = a process that contains database queries and modifications. The statements form a “meaningful whole”.

ACID: atomicity (either all or none of the statements are executed), consistency (the constraints are preserved), isolation (to the user it appears that the transactions run one after another), durability (the modifications of a completed transaction are not lost).

After the execution of the COMMIT SQL statement the transaction can be regarded as final. The modifications of the transaction are finalized. In the case of the ROLLBACK SQL statement the transaction aborts, that is, all statements are rolled back.

One can choose among several isolation levels, regarding what interactions are permitted in the case of transactions happening at the same time.

1. **SERIALIZABLE**: the ACID properties hold for it. The user cannot see a state during another's transaction.
2. **REPEATABLE READ**: Similar to the read-committed restriction. Here, if we read the data again, then what we saw first we will also see the second time. But after the second and subsequent reads we may even see more rows.
3. **READ COMMITTED**: can see only committed data, but not necessarily always the same data.
4. **READ UNCOMMITTED**

2.2.2 DDL

SQL also contains a data definition part.

- **CREATE**: creation
- **DROP**: it drops the entire description, and everything that was connected to it becomes inaccessible.
- **ALTER**: modification of the description

A *constraint* is a relationship between data elements that the DB system has to maintain. It can be a key constraint, or one relating to values or to rows. Modification of a constraint with the **CONSTRAINTS** keyword, standalone constraints: **CREATE ASSERTION** <name> **CHECK** (<condition>). Here basically it has to be checked before any modification of the database. A smart system recognizes which changes may affect which constraints.

A foreign key (from R to S) constraint has to be preserved; this can be violated in two ways: 1. At an insertion into R or a modification in R we specify a value not appearing in S. 2. A deletion or modification in S results in "dangling" rows in R. It can be protected in several ways: by default it does not execute it; in the case of cascading we adjust the table's values to the change; and in the case of set **NULL** we set the affected rows to **NULL**.

For a row constraint, within the **CREATE** statement we can put a **CHECK** (<condition>) statement at the end.

Triggers are executed when some specified event happens, such as the insertion of rows into a table. With standalone constraints (assertions) we can describe a lot of things, but their checking can be a problem. The checking of constraints relating to attributes and rows is simpler (we know when it happens), but with these we cannot express everything. In the case of triggers the user says when a constraint should be checked. Triggers are sometimes also called ECA rules (event-condition-action).

- **Event**: generally some modification in the database, **INSERT**, **DELETE**, **UPDATE**. When?: **BEFORE**, **AFTER**, **INSTEAD**. What?: **OLD ROW**, **NEW ROW**, **FOR EACH ROW**, **OLD/NEW TABLE**, **FOR EACH STATEMENT**
- **WHEN condition**: any SQL true-false-(unknown) condition.
- **Action**: SQL statement, **BEGIN..END**, **PSM** stored procedure

Triggers differ from the previous constraints in three things:

- The system checks triggers only when certain events occur. The permitted events are generally an insertion, deletion, modification relating to a given relation, or the completion of the transaction.
- Instead of immediately preventing the triggering event, the trigger first examines a condition.
- If the trigger's condition is satisfied, then the system executes the action belonging to the trigger. This operation can then prevent the triggering event from happening, or undo it.

View tables: A view table is a relation that we define using stored tables (base tables) and other view tables. There are two kinds: 1. **virtual** = not stored in the database; only the query specifying the relation. 2. **Materialized** = it is computed and then stored. **CREATE** [**MATERIALIZED**] **VIEW** <name> **AS** <query>. View tables can be queried just like base tables. Through view tables the base tables can in some cases also be modified, if the system can carry the modifications through. A virtual view cannot be modified, because it does not exist, but with an **INSTEAD OF** trigger we can still carry out the changes.

3 The procedural extension of SQL (PL/SQL or PSM)

3.1 PSM

When we use SQL statements as part of an application, in a program, the following problems can arise:

- Use of shared variables: common variables between the language and the SQL statement (usable in an SQL statement wherever an expression can be used).
- The problem of type mismatch: The core of SQL is formed by the relational data model. A table – collection, multiset of rows, does not occur as a data type in high-level languages. How can the result of the query be used? We distinguish three cases depending on whether the result of the SELECT FROM [WHERE etc] query returns a scalar value, a single row, or a list (multiset). In the latter case the use of a cursor, traversing the result row by row. For a single row SELECT $e_1, \dots, e_n$ INTO $var_1, \dots, var_n$

We can approach this from a programming point of view in three ways:

1. Extension of SQL with procedural tools, with code segments stored as part of the database schema, with stored modules (e.g. PSM = Persistent Stored Modules, Oracle PL/SQL).
2. Embedded SQL (specific prior embedding EXEC SQL. - A preprocessor transforms it into the host language, e.g. C)
3. Call-level interface: we program in a traditional language, using a function library to access the database (e.g. CLI = call-level interface, JDBC, PHP/DB)

PSM: Persistent Stored Procedures. It consists of a mixture of SQL statements and conventional elements (if, while, etc.). One can also do things that cannot be done in SQL by itself.

<pre> CREATE PROCEDURE eljárás-név (paraméter-lista) [DECLARE ... deklarációk] BEGIN az eljárás utasításai; END;</pre>	<pre> CREATE FUNCTION függvény-név (paraméter-lista) RETURNS értéktípus [DECLARE ... deklarációk] BEGIN utasítások ... END;</pre>
--	--

Figure 14: Structure of a PSM stored procedure and function

We can also give parameters to the stored procedure; these can be of IN, OUT and INOUT mode. Besides these, the name and type of the parameter also have to be specified, a mode-name-type triple. For stored functions we can only have IN-mode parameters.

Some more important statements:

- Calling a procedure is done with the CALL <procedure name> (<argument list>) statement.
- For a function the RETURN statement determines the type of the return value. It is important that this statement does not terminate the function.
- To declare a variable we use this: DECLARE <name> <type>.
- assignment: SET <variable> = <expression>.
- The statements are between BEGIN...END, we separate them with semicolons.
- label: can be given to a statement, by writing a name and a colon before it.

- SQL statements, DML, MERGE.
- exiting from a loop: LEAVE <loop label>
- besides the WHILE-DO loop there is REPEAT-UNTIL

Exceptions: indicated by a 5-character SQLSTATE string. Not always an error; it can also be behavior deviating from the normal. DECLARE <where to go> HANDLER FOR <condition list> <statement>. The <where to go> can be CONTINUE, EXIT, UNDO.

Cursors: If the result of the SELECT returns several rows, then we traverse the rows of the result with some loop. The cursor is essentially a tuple variable that goes through every tuple of a query's result. DECLARE <row pointer> CURSOR FOR (<query>); To use the cursor the OPEN <row pointer>; statement is needed, the effect of which is that the system evaluates the query, and the result of the query becomes accessible; for this traversal a loop has to be started, and the row pointer points to the first row of the result. When we are finished, we close the cursor with the CLOSE <row pointer>; statement.

I: LOOP

A következő tuple lekérése a következő utasítással:

FETCH FROM sormutató **INTO** v1, ...,vn;

A v-k változók listája, a tuple minden komponenséhez van egy.
A kurzor automatikusan a következő kurzorra ugrik.

IF „ellenőrzés: kaptunk-e új sort?”

THEN LEAVE I

END IF;

ENDLOOP:

Figure 15: Structure of a PSM cursor loop

Exiting from the loop is the tricky point of the cursor. The solution is the following: we declare a boolean condition that is true if the SQLSTATE takes a specified value (02000, no next tuple found).

3.2 PL/SQL

In PL/SQL one can not only store procedures and functions, but also run them from the “generic query interface” (sqlplus), like any SQL statement. Triggers are part of PL/SQL.

The declaration part is separated from the body and is optional, and the DECLARE keyword appears only once at the beginning of the part, not before every variable. Assignment with the := sign. A further difference compared to PSM is that for parameters the order is name-mode-type, and the IN OUT mode is written in two words. There are also several new types, e.g. NUMBER can be INT or REAL. One can also refer to the type of an attribute: R.x%TYPE. R%ROWTYPE also exists, which returns a tuple. We obtain the value of component a of tuple x this way: x.a. Instead of ELSEIF (in PSM) it is ELSIF. Instead of LEAVE loop it is EXIT WHEN <condition>.

```

CREATE OR REPLACE PROCEDURE
  <name> (<arguments>) AS
  <optional declarations>
  BEGIN
    <PL/SQL statements>
  END;
.
run

```

szükséges

Eltároljuk az adatbázisban.
Nem igazán futtatja.

Figure 16: Structure of a PL/SQL procedure

Cursor: `CURSOR <name> IS <query>`. In the loop, fetching the cursor's tuple: `FETCH <cursor name> INTO <variable(s)>`. In PL/SQL we omit the cursor's loop this way: `EXIT WHEN <cursor name>%NOTFOUND`.

4 Designing relational database schemas, normal forms, decompositions

4.1 Designing relational database schemas

Dependencies: functional, multivalued; these are used in design (database systems do not support them, there are constraints there).

Normalization: decomposition into good schemas, functional dependencies $\rightarrow$ (1,2,)3NF, BCNF; multivalued dependencies $\rightarrow$ 4NF.

4.1.1 Functional dependencies

$X \rightarrow Y$ is a constraint on the relation R , according to which if two rows agree on all attributes of X , they also have to agree on the attributes of Y .

Notation: $X, Y, Z, \dots$ denote attribute sets; $A, B, C, \dots$ denote attributes. Instead of the attribute set A, B, C we write ABC .

Definition. Let $R(U)$ be a relation schema, and let X and Y be subsets of the attribute set U . Y *functionally depends* on X (in notation $X \rightarrow Y$), if for any table T over R , whenever two rows agree on X , then they also agree on Y , i.e. for $\forall t_1, t_2 \in T, (t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y])$. This essentially means that the value of the attributes in X unambiguously determines the value of the attributes in Y . Notation: $R \models X \rightarrow Y$, that is, R satisfies the dependency $X \rightarrow Y$.

Splitting of right-hand sides: $X \rightarrow A_1 A_2 \dots A_n$ holds for relation R if and only if $X \rightarrow A_1, X \rightarrow A_2, \dots, X \rightarrow A_n$ also hold on R . Example: $A \rightarrow BC$ is equivalent to the pair of dependencies $A \rightarrow B$ and $A \rightarrow C$. There is no rule for splitting left-hand sides, so it is enough to consider the case where a single attribute appears on the right-hand side of the FDs.

Example of functional dependencies: in the Beerdrinkers(name, address, likedBeers, manufacturer, favoriteBeer) table $\text{name} \rightarrow \text{address}$ favoriteBeer (one is the same as $\text{name} \rightarrow \text{address}$ and $\text{name} \rightarrow \text{favoriteBeer}$), $\text{likedBeers} \rightarrow \text{manufacturer}$.

Key, superkey A functional dependency $X \rightarrow Y$ in the special case where $Y = U$ is the key dependency. For a relation schema $R(U)$ a subset K of the attribute set U is a superkey if and only if the FD $K \rightarrow U$ holds. The key can therefore also be defined based on the concept of dependency: we call an attribute set K a key if all the other attributes depend on it, but omitting any attribute from K this no longer holds (that is, a minimal superkey). Example based on the previous one: $\{\text{name}, \text{likedBeers}\}$ is a superkey, these two attributes functionally determine the remaining attributes.

Implication of dependencies: F implies $X \rightarrow Y$ if in every table in which all dependencies of F hold, $X \rightarrow Y$ also holds. Notation: $F \models X \rightarrow Y$, if F implies $X \rightarrow Y$.

Let $X_1 \rightarrow A_1, X_1 \rightarrow A_2, \dots, X_1 \rightarrow A_n$ be given FDs; we would like to know whether $Y \rightarrow B$ holds for relations for which the previous FDs hold. Example: in case $A \rightarrow B$ and $B \rightarrow C$ hold, $A \rightarrow C$ certainly holds. When checking whether $Y \rightarrow B$ holds, take two rows that agree on all attributes in Y . Use the given FDs to prove that the previous two rows also have to agree on other attributes. If B is such an attribute, then $Y \rightarrow B$ holds. Otherwise the previous two rows will give an occurrence that satisfies all the prescribed equalities, yet $Y \rightarrow B$ still does not hold, that is, $Y \rightarrow B$ is not a consequence of the given FDs.

Armstrong axioms: Let $R(U)$ be a relation schema and $X, Y \subseteq U$, and let XY denote the union of the attribute sets X and Y . Let F be an arbitrary set of functional dependencies.

- FD1 (reflexivity): in case $Y \subseteq X, X \rightarrow Y$.
- FD2 (augmentation): in case $X \rightarrow Y$ and for arbitrary $Z, XZ \rightarrow YZ$.
- FD3 (transitivity): in case $X \rightarrow Y$ and $Y \rightarrow Z, X \rightarrow Z$.

The Armstrong axiom system is sound and complete, that is, every derivable dependency is also implied, and those dependencies that F implies can also be derived from F . $F \vdash X \rightarrow Y \iff F \models X \rightarrow Y$

Derivation: $X \rightarrow Y$ is derivable from F if there is a finite derivation $X_1 \rightarrow Y_1, \dots, X_k \rightarrow Y_k, \dots, X \rightarrow Y$ such that for $\forall k, X_k \rightarrow Y_k \in F$ or $X_k \rightarrow Y_k$ can be obtained based on the axioms FD1, FD2, FD3 from the dependencies appearing before it in the derivation. Notation: $F \vdash X \rightarrow Y$, if $X \rightarrow Y$ is derivable from F .

Further derivable rules:

- Union rule: in case $F \vdash X \rightarrow Y$ and $F \vdash X \rightarrow Z, F \vdash X \rightarrow YZ$.
- Pseudotransitivity: in case $F \vdash X \rightarrow Y$ and $F \vdash WY \rightarrow Z, F \vdash XW \rightarrow Z$.
- Decomposition rule: in case $F \vdash X \rightarrow Y$ and $Z \subseteq Y, F \vdash X \rightarrow Z$.

Closure of an attribute set: Given a schema R and a set F of functional dependencies, X^+ is the set of all attributes A for which $X \rightarrow A$ follows from F . For a schema (R, F) let $X \subseteq R$. Definition: $X^{+(F)} := \{A | F \vdash X \rightarrow A\}$ is the closure of the attribute set X with respect to F . Lemma: $F \vdash X \rightarrow Y \iff Y \subseteq X^+$. Its consequence: to solve the implication problem it is enough to compute X^+ efficiently.

The implication can also be decided with closure. Starting point: $Y^+ = Y$. Induction: We look for FDs whose left-hand side is already in Y^+ . If $X \rightarrow A$ is such, we add A to Y^+ .

Projection of FDs: Motivation: “normalization”, during which we can decompose a relation schema into several schemas. Example: $ABCD, F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow A\}$. Let us decompose into ABC and AD . Which FDs hold on ABC ? Not only $AB \rightarrow C$, but also $C \rightarrow A$! Computing the projection: Let us start from the given FDs and find all non-trivial FDs that follow from the given FDs. (Non-trivial = the left side does not contain the right side.) Let us only deal with those FDs in which the attributes of the projected schema appear. Projection of dependencies: Given (R, F) , and for $R_i \subseteq R: \Pi_{R_i}(F) := \{X \rightarrow Y | F \vdash X \rightarrow Y, XY \subseteq R_i\}$.

The multivalued dependency (MVD): $X \twoheadrightarrow Y$ holds over the relation R : if for any two rows that agree on all attributes of X , the values belonging to the attributes of Y can be swapped, that is, the two resulting new rows will be in R . In other words: for every value of X the values belonging to Y are independent of the values of $R - X - Y$. Definition: for $X, Y \subseteq R, Z := R - XY, X \twoheadrightarrow Y$ is a multivalued dependency. The dependency holds in a table if the existence of rows of a certain pattern guarantees the existence of other rows.

Formal definition: A relation r with schema R satisfies the dependency $X \twoheadrightarrow Y$ if for $t, s \in r$ and $t[X] = s[X]$ there

exist $u, v \in r$ such that $u[X]=v[X]=t[X]=s[X]$, $u[Y]=t[Y]$, $u[Z]=s[Z]$, $v[Y]=s[Y]$, $v[Z]=t[Z]$. Claim: It is enough to require the existence of only one of u, v . Example: Beerdrinkers(name, address, phone, likedBeers) table. The phone numbers of the beer drinkers are independent of the beers they like: $\text{name} \twoheadrightarrow \text{phone}$ and $\text{name} \twoheadrightarrow \text{likedBeers}$. Thus every phone number of a given beer drinker stands in combination with every beer they like. This phenomenon is independent of the functional dependencies.

MVD rules:

- Every FD is an MVD. If $X \rightarrow Y$ and two rows agree on X , they also agree on Y , because of which if we swap these, we get back the original rows, that is: $X \rightarrow \rightarrow Y$.
- Complementation: If $X \rightarrow \rightarrow Y$ and Z denotes the set of all the other attributes, then $X \rightarrow \rightarrow Z$.
- It cannot be split into pieces.
- Claim: from $X \rightarrow \rightarrow Y$ it does not follow that $X \rightarrow \rightarrow A$, if $A \in Y$. (The right-hand sides cannot be split apart!)
- Not transitive.

In the definition of losslessness and dependency preservation, now instead of a set F of functional dependencies, a set D of dependencies can also contain multivalued dependencies.

Theorem: $d = (R_1, R_2)$ is a lossless decomposition of R if and only if $D \vdash R_1 \cap R_2 \rightarrow \rightarrow R_1 - R_2$.

4.2 Normal forms

- *Boyce-Codd normal form*: relation R is in BCNF if for every non-trivial FD $X \rightarrow Y$ in R , X is a superkey. Non-trivial: Y is not part of X . Superkey: contains a key (it can itself be a key). If there is a consequence FD in F that violates BCNF, then an FD in F also violates it. We compute X^+ : If not all attributes appear here, X is not a superkey.
- For certain sets of FDs we can lose dependencies during decomposition. In *3rd normal form* (3NF) the BCNF condition is modified so that in the previous case we do not have to decompose. An attribute is a prime attribute if it is an element of at least one key. $X \rightarrow A$ violates 3NF if and only if X is not a superkey and A is not prime.
- The *4th normal form* resembles BCNF, that is, the left-hand side of every non-trivial multivalued dependency is a superkey. The redundancy caused by MVDs is not eliminated by BCNF. The solution: the fourth normal form. In the fourth normal form (4NF), when we decompose, we handle the MVDs like the FDs, but in finding the keys they do not count.

A relation R is in 4NF if: for every non-trivial MVD $X \rightarrow \rightarrow Y$, X is a superkey. Non-trivial MVD: Y is not a subset of X , and X and Y together do not give all the attributes. The definition of superkey remains the same, that is, it only depends on the FDs. Definition: R is in 4NF with respect to D if for $XY \neq R$, $Y \not\subseteq X$, and $D \vdash X \rightarrow \rightarrow Y$, we have $D \vdash X \rightarrow R$.

Definition: a decomposition $d = \{R_1, \dots, R_k\}$ is in 4NF with respect to D if every R_i is in 4NF with respect to $\Pi_{R_i}(D)$.

Claim: If R is in 4NF, then it is also in BCNF. Consequence: There is not always a dependency-preserving and lossless 4NF decomposition.

We can always make a lossless 4NF decomposition, similarly to the naive BCNF decomposition algorithm.

If R is in 4NF, then it is also in BCNF.

Theorems: There is always a lossless (LL) decomposition into BCNF and a lossless dependency-preserving (LL DP) decomposition into 3NF.

4.3 Decompositions

Bad design also results in anomalies. In good design the goal is the elimination of anomalies and redundancy.

név	cím	kedveltSörök	gyártó	kedvencS
Janeway	Voyager	Bud	A.B.	WickedAle
Janeway	???	WickedAle	Pete's	???
Spock	Enterprise	Bud	???	Bud

Figure 17: Beerdrinker(name, address, likedBeers, manufacturer, favoriteBeer) table

- Modification anomaly: if we change Janeway to Jane, do we do this for every row? We change one occurrence of a piece of data, but not its other occurrences.
- Deletion anomaly: If nobody likes the Bud beer, then with that we also delete the info about who manufactured it. At deletion we also lose data that we did not want to.
- Insertion anomaly: and to enter such a manufacturer? A constraint, trigger is needed so that we can check (e.g. the key dependency).

Decomposition (splitting): We can get rid of the above problems with decomposition (splitting)! Definition: $d = \{R_1, \dots, R_k\}$ is a decomposition of (R, F) if no attribute is left out, that is, $R_1 \cup \dots \cup R_k = R$. (The splitting of the data table is done with projection.)

Expectations:

1. The decomposition should be lossless, that is, there should be no loss of information. With the above notation: if $r = \Pi_{R_1}(r) \bowtie \dots \bowtie \Pi_{R_k}(r)$ holds, then we say of the previous join that it is lossless. Here r denotes a relation with schema R . Chase test for losslessness: We make a decomposition. From the join of the elements of the decomposition we take a row. With the algorithm we prove that this row is also a row of the original relation.
2. The projections should have good properties, and the dependency system of the projection should be simple (normal forms: BCNF, 3NF def. later)
3. Preservation of dependencies in the projections (DP) From the dependencies valid in the decompositions, all the dependencies imposed on the original schema should follow. Given (R, F) , $d = \{R_1, \dots, R_k\}$ is a dependency-preserving decomposition if and only if every dependency in F is derivable from the projected dependencies: for every $X \rightarrow Y \in F$, $\Pi_{R_1}(F) \cup \dots \cup \Pi_{R_k}(F) \vdash X \rightarrow Y$.

From dependency preservation losslessness does not follow, and from losslessness dependency preservation does not follow.